

MODEL QUESTION PAPER-01 - 2023-24

(BCS402)

MICROCONTROLLERS

"Start where you are,
 Use what you have,
 Do what you can"

- Q.1. a) Mention the difference between
 1) Microcontroller and Microprocessor
 2) RISC and CISC

MICROPROCESSORS versus MICROCONTROLLERS:

A microprocessor is an electronic component that is used by a computer to do its work. It is a central processing unit on a single integrated circuit chip containing millions of very small components including transistors, resistors and diodes that works together.

A microcontroller is a compact integrated circuit designed to govern a specific operation in an embedded system. A typical microcontroller includes a processor, memory and input/output (I/O) peripheral on a single chip.

Microprocessors

- Microprocessors generally does not have RAM, ROM and I/O pins
- Microprocessor usually use its pins as a bus to interface to RAM, ROM, and peripheral devices. Hence, the controlling bus is expandable at the board level
- Microprocessors are generally capable of being built into bigger purpose applications
- Microprocessors, generally do not have power saving system
- The overall cost of systems made with microprocessors is high, because of the high number of external components required.

Microcontrollers

- Microcontroller is "all in one" processor, with RAM, I/O ports, all on the chip
- Controlling bus is internal and not available to the board designer.
- Microcontrollers are usually used for more dedicated applications
- Microcontrollers have power saving system, like idle mode or power saving mode so overall it uses less power.
- Microcontrollers are made by using complementary metal oxide semiconductor technology, so they are far cheaper than microprocessors

→ processing speed of general microprocessor is above 1GHz, so it works much faster than microcontrollers.

→ processing speed of microcontroller is about 8 MHz to 50 MHz

→ Microprocessors are based on von-Neumann model, where, program and data are stored in same memory module

→ Microcontrollers are based on Harvard architecture, where program memory and data memory are separate.

CISC

RISC

→ Complex instructions, taking multiple clock

→ Simple instructions, taking single clock

→ Emphasis on hardware, Complexity is in the micro-program/processor

→ Emphasis on software, Complexity is in the Compiler

→ Complex instructions, instructions executed by

→ Reduced instructions, instructions executed by hardware

→ Variable format instruction, single register set and many instructions

→ Fixed format instructions, multiple register sets and few instructions

M T W T F S S
☐ ☐ ☐ ☐ ☐ ☐ ☐

COMPASS

Date : _____

→ Many instructions and many addressing modes

→ Fixed instructions and few addressing modes

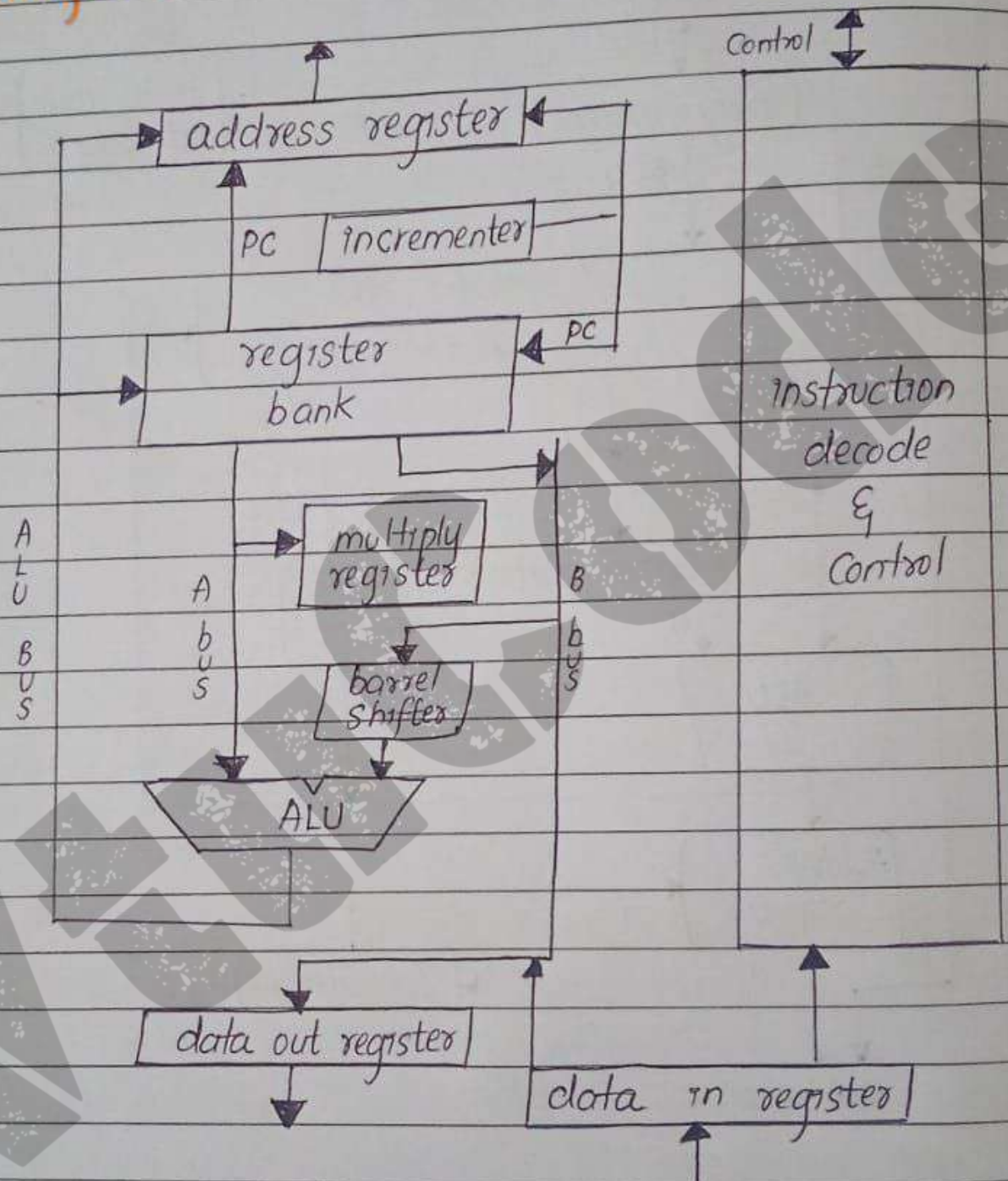
→ Conditional jump is usually based on status register bit

→ Conditional jump can be based on a bit anywhere in memory

→ Memory reference is embedded in many instructions

→ Memory reference is embedded in LOAD/STORE instructions.

1. b) Explain the architecture of an ARM embedded device with the help of neat diagram.



When we use the term architecture, we mean a functional specification. In the case of the Arm architecture, we mean a functional specification for a processor. An architecture

Specifies how a processor will behave, such as what instructions it has and what the instruction do.

You can think of an architecture as an contract between the hardware and the software. The architecture describe what functionality the software can rely on the hardware to provide.

The architecture specifies:

Instruction set

- The function of each instruction
- How that instruction is represented in memory

Register set

- How many registers there are
- The size of the registers
- The function of the registers
- Their initial state

Exception model

- The different levels of privilege
- The types of exceptions
- What happens on taking or returning from an exception

Memory model

- How memory accesses are ordered
- How the caches behave, when and how software must perform explicit maintenance.

Debug, trace, and profiling

- How breakpoints are set and triggered.

M T W T F S S
☐ ☐ ☐ ☐ ☐ ☐ ☐

COMPASS
Date : _____

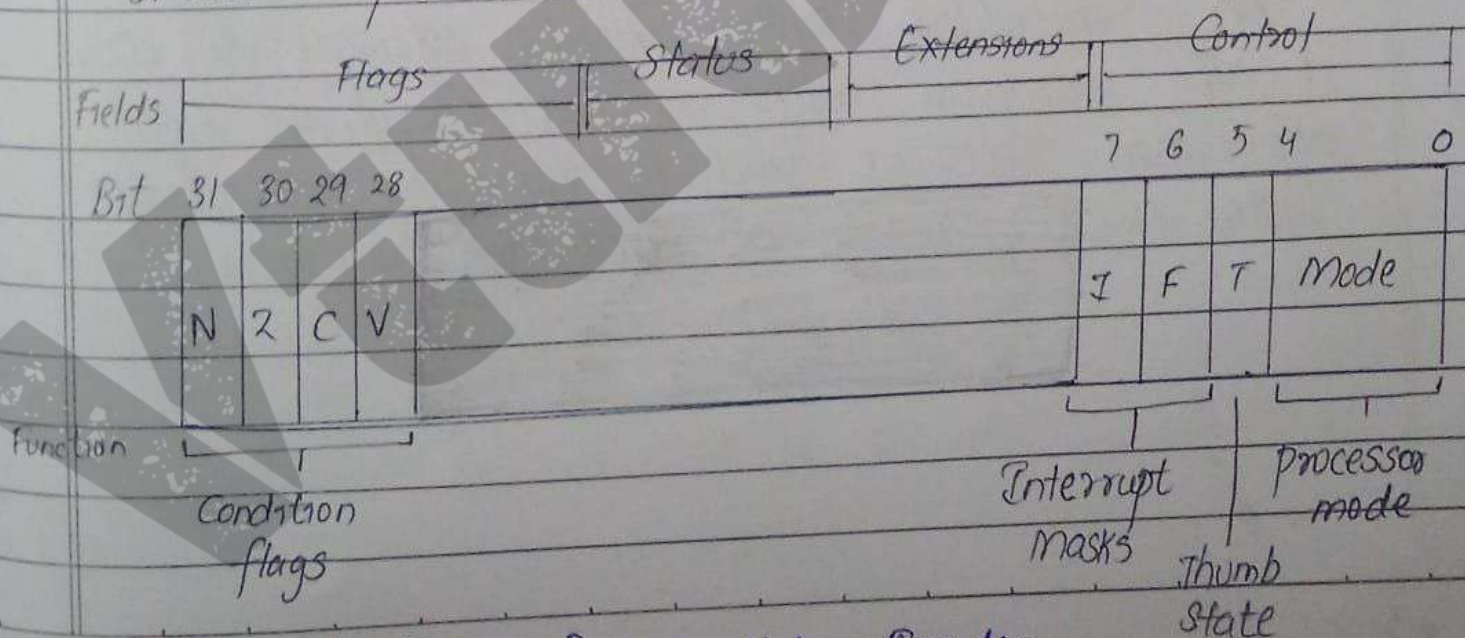
Micro-architecture includes thing like:

- * Pipeline length and layout
- * Number and sizes of caches
- * Cycle counts for individual instructions
- * Which optional features are implemented

OR

Q1. Explain in details about current program status Register (CPSR).

The ARM core uses the cpsr to monitor and control internal operations. The cpsr is a dedicated 32-bit register and resides in the register file. The following figure shows the basic layout of a generic program status register. Note that the shaded parts are reserved for future expansion.



A Generic Program Status Register

The cpsr is divided into four fields, each 8 bits wide: flags, status, extensions, and control. In current design the extension and status fields are reserved for future use.

→ The 'Control field' contains the processor mode, state, and interrupt mask bits

→ The 'flag field' contain the condition flags.

Some ARM processor cores have extra bits allocated. For example, the J bit, which can be found in the flags field, is only available on Jazella-enabled processors, which execute 8-bit instructions.

It is highly probable that future designs will assign extra bits for the monitoring and control of new features.

2. b. with a neat diagram, explain embedded System Hardware.

EMBEDDED SYSTEM HARDWARE :-

Embedded systems can control many different devices, from small sensors found on a production line, to the real-time control systems used on a NASA space probe. All these devices use a combination of software and hardware components.

The following Figure shows a typical embedded device based on an ARM core. Each box represents a feature or functions. The lines connecting the boxes are the buses carrying data.

M T W T F S S
☐ ☐ ☐ ☐ ☐ ☐ ☐

COMPASS

Date : _____

2. 'Controllers' coordinates important functional blocks of the system. Two commonly found controllers are interrupt and memory controllers.
3. The 'peripherals' provide all the input-output capability external to the chip and are responsible for the uniqueness of the embedded device.
4. A 'bus' is used to communicate between different parts of the device.

MODULE-2

3 a) Explain different arithmetic instructions in ARM processor with an example.

ARITHMETIC INSTRUCTIONS:

- The arithmetic instructions implement addition and subtraction of 32-bit signed and unsigned values.

Syntax: $\langle \text{instruction} \rangle \{ \langle \text{cond} \rangle \} \{ \text{S} \} \text{Rd}, \text{Rn}, \text{N}$

ADC	add two 32-bit values and carry	$\text{Rd} = \text{Rn} + \text{N} + \text{Carry}$
ADD	add two 32-bit values	$\text{Rd} = \text{Rn} + \text{N}$
RSB	reverse subtract of two 32-bit values	$\text{Rd} = \text{N} - \text{Rn}$
RSC	reverse subtract with carry of two 32-bit values	$\text{Rd} = \text{N} - \text{Rn} - !(\text{carry flag})$
SBC	subtract with carry of two 32-bit values	$\text{Rd} = \text{Rn} - \text{N} - !(\text{carry flag})$
SUB	subtract two 32-bit values	$\text{Rd} = \text{Rn} - \text{N}$

- In the following example, subtract instruction subtracts a value stored in register $r2$ from a value stored in the register $r1$. The result is stored in register $r0$.

PRE $r0 = 0x00000000$

$r1 = 0x00000002$

$r2 = 0x00000001$

SUB $r0, r1, r2$

POST $r0 = 0x00000001$

- In the following example, the reverse subtract instruction (RSB) subtract $r1$ from the constant value #0. Writing the result is $r0$.

PRE $r0 = 0x00000000$

$r1 = 0x00000077$

RSB $r0, r1, \#0$; $Rd = 0x0 - r1$

POST $r0 = -r1 = 0xffffffff89$

- The use of inline barrel shifter with an arithmetic instruction. The instruction multiplies the value stored in register 1 by three.
- Register $r1$ is first shifted one location to the left to give the value of twice $r1$. The ADD instruction then adds the result of the barrel shift operation to register $r1$. The final result transferred into register $r0$ is equal to three times the value stored in register $r1$.

PRE $r0 = 0x00000000$

$r1 = 0x00000005$

ADD $r0, r1, r1, LSL \#1$

POST $r0 = 0x0000000f$

$r1 = 0x00000005$

b. Explain Single register load store addressing mode
Syntax, table, index with an example

Single-Register Load-store Addressing Modes

- The ARM instruction set provides different modes for addressing memory
- These modes incorporate one of the indexing methods: preindex with writeback

Index method	Data	Base address register	Example
Preindex with writeback	$mem[base + offset]$	$base + offset$	$LDR\ r0, [r1, \#4]!$
Preindex	$mem[base + offset]$	not updated	$LDR\ r0, [r1, \#4]$
postindex	$mem[base]$	$base + offset$	$LDR\ r0, [r1], \#4$

Example :-

PRE

$r0 = 0x00000000$

$r1 = 0x00090000$

$mem_{32}[0x00090000] = 0x01010101$

$mem_{32}[0x00090004] = 0x02020202$

LDR $r0, [r1, \#4]!$

Pre indexing with writeback:

POST(1) $r0 = 0x02020202$
 $r1 = 0x00009004$
 LDR $r0, [r1, \#4]$

Preindexing:

POST(2) $r0 = 0x02020202$
 $r1 = 0x00009000$
 LDR $r0, [r1], \#4$

Postindexing

POST(3) $r0 = 0x01010101$
 $r1 = 0x00009004$

OR

407 Explain barrel shifter instructions in ARM with suitable example.

→ A barrel shifter is a feature of the ARM processor that can shift or rotate the binary pattern in one of the source registers left or right by a specific number of positions.

→ The barrel shifter can perform different types of operation, such as logical shift, arithmetic shift, rotate, and rotate with extend.

→ The barrel shifter can be used by data processing and data transfer instructions to execute complex operations in a single instruction.

→ Example below illustrate the use of the inline barrel shifter with an arithmetic instruction. The instruction multiplies the value stored in register $r1$ by three.

→ Register $r1$ is first shifted one location to the left to give the value of twice $r1$. The ADD instruction then adds the result of the barrel shift operation to register $r1$. The final result transferred into register $r0$ is equal to three times the value stored in register $r1$.

```
PRE    r0 = 0x00000000
        r1 = 0x00000005
        ADD    r0, r1, r1, LSL #1
```

```
POST   r0 = 0x0000000f
        r1 = 0x00000005
```

b) Explain different Logical instructions in ARM processor with an example.

Logical instructions perform bitwise operations on the two source registers.

Syntax: <instruction> {<cond>} {s} Rd, Rn, N

Mnemonic	operation	Comment
AND $r0, r1, r2$	$r0 := r1 \& r2$	And two 32-bit values
ORR $r0, r1, r2$	$r0 := r1 r2$	OR of two 32-bit values
EOR $r0, r1, r2$	$r0 := r1 \wedge r2$	Exclusive OR of two 32-bit values
BIC $r0, r1, r2$	$r0 := r1 \& \sim r2$	Logical bit clear AND NOT

The AND, OR and Ex-OR are trivial logical operations. Let us consider the execution of BIC instruction. Assume pre-execution nibble values for simplicity, let $r1 = 0b1111$ and $r2 = 0b1010$. post execution of the instruction

BIC $r0, r1, r2$;

$r0 = 0b0101$. The implication of the result is that every binary one in $r2$ register clears a corresponding bit location in $r0$ register.

Example:- a logical OR operation between register $r1$ & $r2$ and the result is in $r0$.

PRE $r0 = 0x00000000$

$r1 = 0x02040608$

$r2 = 0x10305070$

ORR $r0, r1, r2$

POST $r0 = 0x12345678$

MODULE-3

5) Explain code optimization, profiling and cycle counting.

C Compilers and Optimization:

Optimizing code takes time and reduces source code readability. therefore it's only worth optimizing functions that are frequently executed and important for performance. C Compiler have to translate our C function literally into assembler so that it works for all possible inputs.

To write efficient C code, we must be aware of areas where the C Compiler has to be conservative, the limits of the processor architecture the C compiler is mapping to, and the limits of a specific C Compiler.

Most Common C Compilers are:

- armcc from ARM Developer Suite version 1.1 (ADS1.1)
- arm-elf-gcc version 2.95.2. This is the ARM target for the GNU C Compiler, gcc.

Profiling and cycle counting:

- A profiler is a tool that measures the proportion of time or processing cycle spent in each subroutine

- A cycle counter measures the number of cycles taken by a specific routine.
- The ARM simulator used by the ADS 1.1 debugger is called the ARMulator and provides profiling and cycle counting features.
- The ARMulator profiler works by sampling the program counter pc at regular intervals.
- ARM implementations do not normally contain cycle-counting hardware, so to easily measure cycle counts we should use an ARM debugger with ARM simulator. We can configure the ARMulator to simulate a range of different ARM cores and obtain cycle count benchmarks for a number of platforms.

b) Write a C program that prints the square of integers between 0 to 9 using function and explain how to convert this C function to an assembly function with command.

C program

```
#include <stdio.h>
void print_squares();
int main() {
    print_squares();
    return 0;
}
```



```
// function to print square of integers from 0 to 9
void print_squares() {
    for (int i=0; i<10; i++) {
        int square = i*i;
        printf("The square of %d is %d\n", i, square);
    }
}
```

Assembly language program

```
.file "square.c"
.text
.globl print_squares
.type print_squares, @function
```

print_squares:

```
push r1 ; save base pointer (rbp) as r1
mov r0, r2 ; move the value of rax to r0 (rbp)
mov r2, 0WORD PTR [rip+100h]; Load address of format
string into r2
```

```
mov r3, 0 ; Initialize loop counter r3 to 0
```

```
jmp .L2
```

.L4:

```
mov r0, r3 ; Move loop counter (r3) to r0
```

```
imul r0, r3 ; Compute square: r0 = r0 * r3
```

```
mov r2, r0 ; move the square to r2
```

```
mov r0, r2 ; move address of format string to r0
```

```
add r3, 1 ; Increment loop counter
```

.L2:

```
cmp r3, 10 ; Compare loop counter with 10
```

```
jlt .L4 ; If less than 10, loop again
```

```
pop r1 ; Restore base pointer
```

```
ret
```


Steps in Assembly language

Function Prologue:

- In assembly, we need to set up the
- stack frame (if required) and save any necessary registers.
- Allocate space for local variables

Function Body

- Load the input argument (integer) from the appropriate register (e.g., R0 on ARM)
- multiply the input by itself to calculate square. store the result in the appropriate register

Function Epilogue:

- Restore any saved registers.
- Clean up the stack frame (if used).

Return:

- Return the result (square) to the caller (usually via a specific register).

OR

6a) Discuss how registers are allocated to optimize the program.

The compiler attempts to allocate a processor register to each local variable we use in a function. It will try to use the same register for different

local variables if the use of the variables do not overlap. When there are more local variables than available registers, the compiler stores the excess variables on the processor stack. These variables are called spilled or swapped out variables since they are written out to memory.

Spilled variables are slow to access compared to variables allocated to registers.

To implement a function efficiently, we need to

- minimize the number of spilled variables
- ensure that the most important and frequently accessed variables are stored in registers.

In theory, the C Compiler can assign 14 variable to registers without spillage. In practice, some compilers use a fixed register such as r12 for intermediate scratch working and do not assign variable to this register. Also, complex expressions require intermediate working register to evaluate. Therefore, we should try to limit the internal loop of functions to using at most 12 local variables.

If the Compiler does need to swap out variable, then it chooses which variables to swap out based on frequency of use.

Summary: -

Efficient Register Allocation:

- * Limit the number of local variables in the internal loop of functions to 12. The compiler should be able to allocate these to ARM registers.
- * We can guide the compiler as to which variables are important by ensuring these variables are used within the innermost loop.

b) Develop an ALP to find the sum of first 10 integer number.

; program to find sum of first 10 integers.
 AREA SUM10, CODE, READONLY

START

MOV R0, #10

MOV R1, #00

LOOP

ADD R1, R1, R0

SUBS R0, R0, #1

BNE LOOP

STOP B STOP

END

Input:

$$7 \times 6 \times 5 \times 4 \times 3 \times 2 \times 1 = 55$$

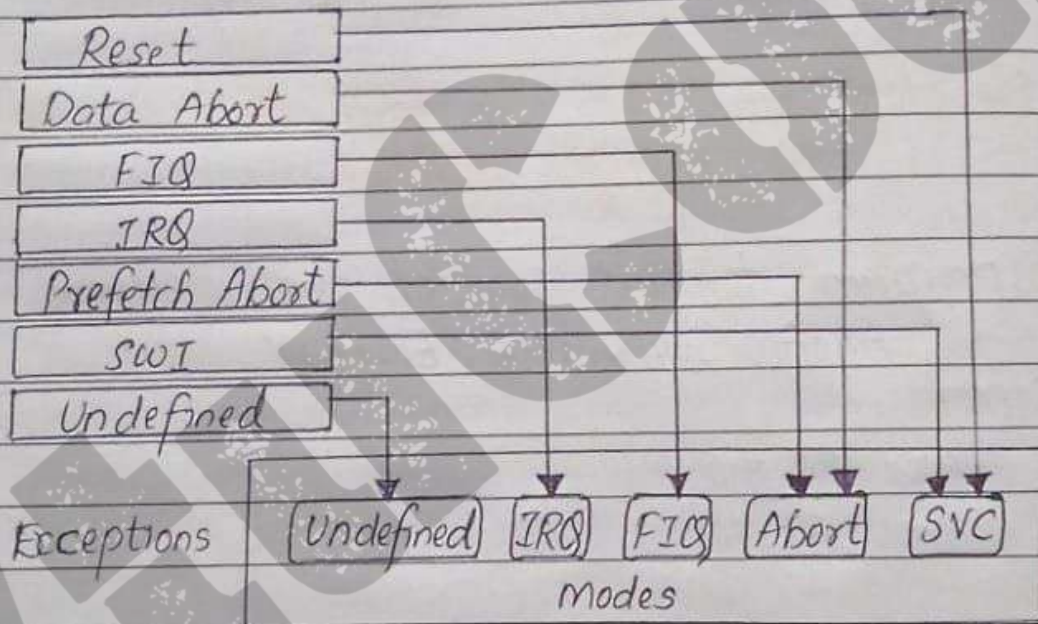
Output:

R1 = 0x00000037
 Program: @vtu23

MODULE-4

7) a) With a neat diagram explain ARM processor exceptions and modes.

Whenever an exception occurs, the core enters a specific mode. The ARM processor modes can be entered manually by changing the cpsr. When an exception occurs the ARM processor always switches to ARM state.



Exceptions and associated modes.

The user and system mode are the only two modes that are not entered by an exception when an exception causes a mode change, the core automatically

- Saves the cpsr to the spsr of the exception mode
- Saves the pc to the lr of the exception mode

- Sets the cpsr to the exception mode.
- Sets pc to the address of the exception handler

Explain assigning interrupts and interrupt latency.

Assigning Interrupts

A System designer can decide which hardware peripheral can produce which interrupt request. This decision can be implemented in hardware or software (or both) and depends upon the embedded system being used. An interrupt controller unit is used to connect multiple external interrupt to one of the two ARM interrupt requests either IRQ or FIQ. The system designers will use a standard design practice to assigning interrupts.

- Software Interrupts are normally reserved to call privileged operating system routines. For example, an SWI instruction can be used to change a program running in user mode to a privileged mode.
- IRQ Requests are normally assigned for general-purpose interrupts. The IRQ exception has a lower priority and higher interrupt latency than the FIQ exception.
- Fast Interrupt Requests are normally reserved for a single interrupt source that requires a fast response time.

→ In an embedded operating system design, the FIQ exception is used for a specific application and the IRQ exception are used for more general operating system activities.

Interrupt Latency

It is the time interval, from an external interrupt request signal being raised to the first fetch of an instruction of a specific interrupt service routine (ISR).

Interrupt latency depends on a combination of hardware and software. System design must balance the system design to handle multiple simultaneous interrupt sources and minimize interrupt latency.

If the interrupts are not handled in a timely manner, then the system will exhibit slow response times.

Software handler's have two main methods to minimize interrupt latency.

- 1) Nested interrupt handler
- 2) prioritization.

Nested interrupt handler:

Nested interrupt handler allows other interrupts to occur even when it is currently servicing an existing interrupt.

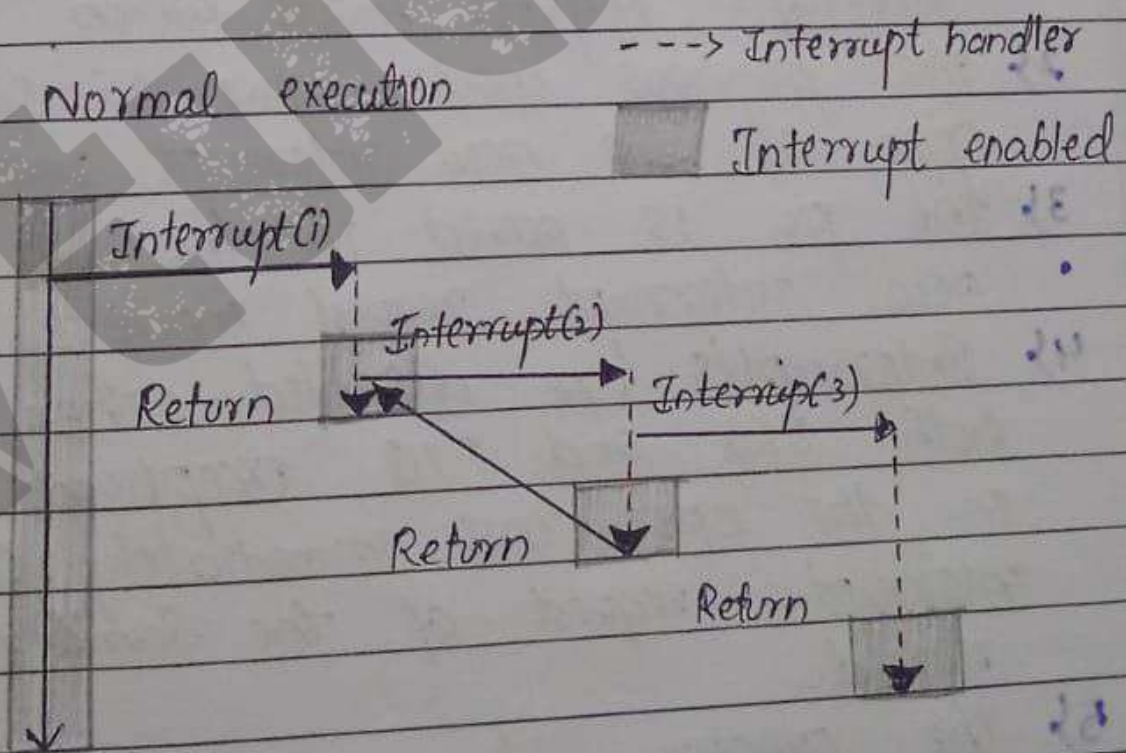
prioritization

We can program the interrupt controller to ignore interrupts of the same or lower priority than the interrupt we are handling presently, so only a higher-priority task can interrupt our handler. We then re-enable the interrupts.

The processor spends time in the lower-priority interrupts until a higher-priority interrupt occurs.

Therefore higher-priority interrupts have a lower average interrupt latency than the lower-priority interrupts.

It reduces latency by speeding up the completion time on the critical time-sensitive interrupts.



A three-level nested interrupt

OR

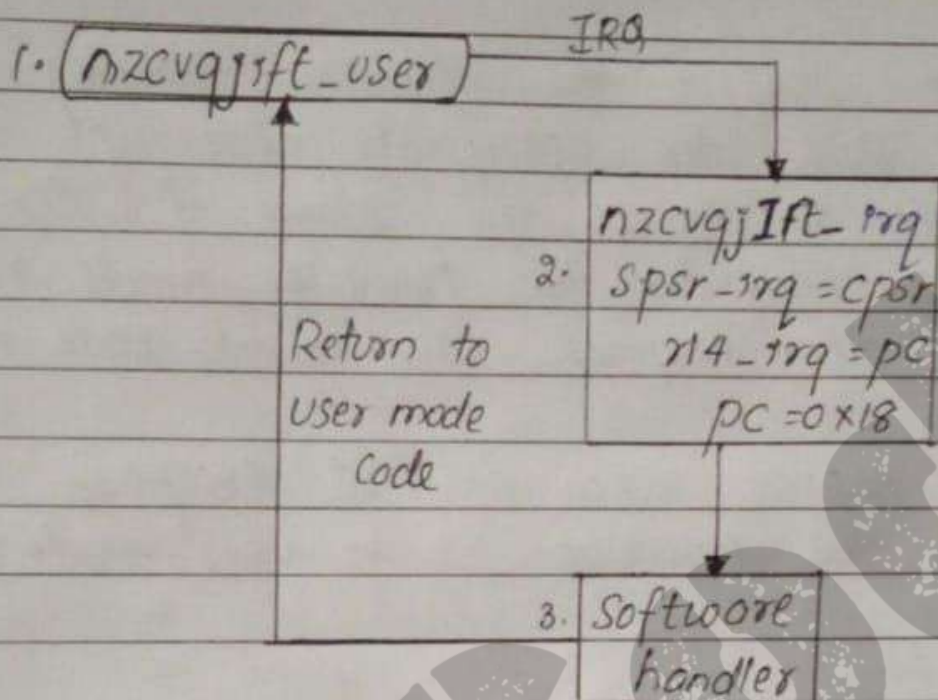
8) a) Briefly explain what happens when an IRQ and FIQ exception is raised with an ARM processor

IRQ and FIQ exceptions only occur when a specific interrupt mask is cleared in the cpsr. The ARM processor will continue executing the current instruction in the execution stage of the pipeline before handling the interrupt.

An IRQ or FIQ exception causes the processor hardware to go through a standard procedure listed below,

- 1) The processor changes to a specific interrupt request mode, which being raised.
- 2) The previous mode's cpsr is saved into the spsr of the new interrupt request mode.
- 3) The pc is saved in the lr of the new interrupt request mode.
- 4) Interrupts are disabled - either the IRQ or both IRQ and FIQ exceptions are disabled in the cpsr. This immediately stops another interrupt request of the same type being raised.
- 5) The processor branches to a specific entry in the vector table.

what happens when an IRQ exception is raised when the processor is in user mode?



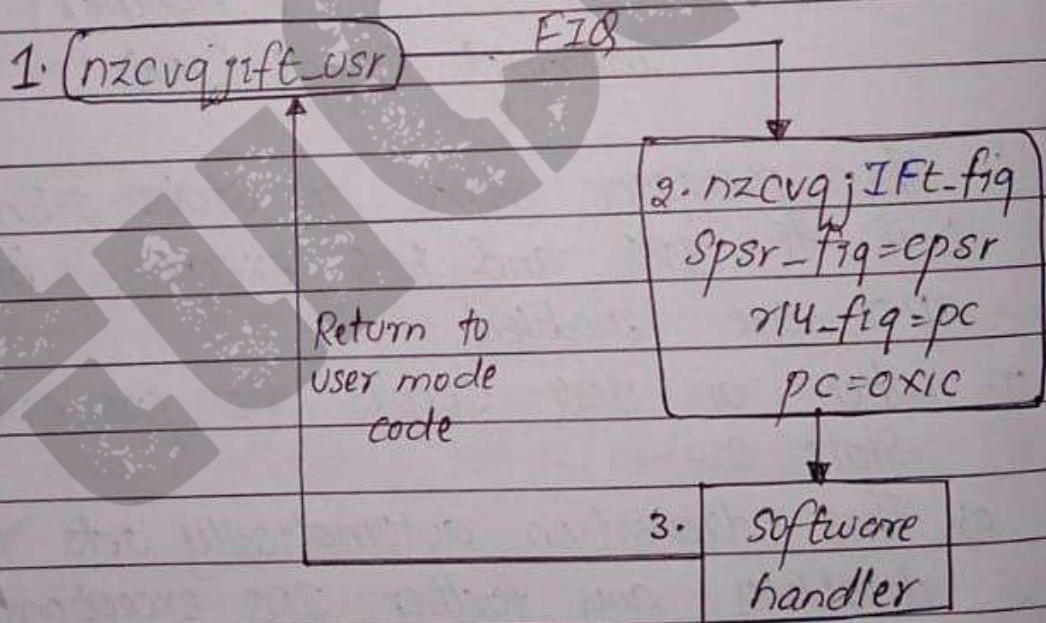
Interrupt Request (IRQ)

- i. The processor starts in State 1. In this mode both the IRQ and FIQ exception bits in the cpsr are enabled.
- ii. When an IRQ occurs the processor moves into State 2 ->
 - a. This transition automatically sets the IRQ bit to one, disabling any further IRQ exceptions.
 - b. The FIQ exception remains enabled because FIQ has a higher priority and does not get disabled when a low-priority IRQ exception is raised.
 - c. The cpsr processor mode changes to IRQ mode.
 - d. The user mode cpsr is automatically copied into SPSR-irq.
 - e. Register R14-irq is assigned to value of the PC when the interrupt was raised.

fy The pc is then set to the IRQ entry $\pm 0x18$ in the vector table.

iii In state 3 the software handler takes over and calls the appropriate interrupt service routine to service the source of the interrupt. After completion, the processor mode reverts back to the original user mode code in state 1.

What happens when an FIQ exception is raised when the processor is in user mode?



iy The processor starts in state 1. In this mode both the IRQ and FIQ exception bits in the cpsr are enabled.

iiy when an FIQ occurs the processor moves into state 2 ->

- a) This transaction automatically sets the IRQ bit and FIQ to one, disabling both IRQ and FIQ exceptions,
- b) The cpsr processor mode changes to FIQ mode,
- c) The user mode cpsr is automatically copied into spsr_fiq,
- d) Register r14_fiq is assigned the value of the pc when the interrupt was raised,
- e) The pc is then set to the FIQ entry + 0x1c in the vector table.
- iii) In state 3 the Software handler takes over and calls the appropriate interrupt services routine to service the source of the interrupt. After completion, the processor mode reverts back to the original user in state 1.
- iv) When processor changes from user mode to FIQ mode, there is no requirement to save registers r8 to r12 since these registers are banked in FIQ mode. These registers can be used to hold temporary data, such as buffer pointers or counters. This makes FIQ ideal for servicing a single-source, high-priority, low-latency interrupt.

b) Explain firmware execution flow and explain Red Hat RedBoot.

Firmware is a type of software that is embedded into hardware devices to control their

operations. The execution flow of firmware typically involves the following steps.

1. Power-On/Reset: The processor starts executing code from a fixed location.
2. Bootloader Initialization: Initializes hardware and prepares to load the main firmware.
3. Hardware Initialization: Sets up memory, peripherals, and performs self-checks.
4. Firmware Loading: Loads the main firmware from non-volatile memory.
5. Main Firmware Execution: Controls the device's primary functions.
6. System Operation: Manages ongoing tasks and user interactions.
7. Updates: Firmware may be updated periodically.

Red Hat RedBoot

1. Bootloader: Initializes hardware and loads the operating system or application.
2. Development: Facilitates debugging and testing with a command-line interface.
3. Network Support: Allows network-based firmware loading and updates.
4. Customization: Tailors features to specific hardware needs.

Module-5

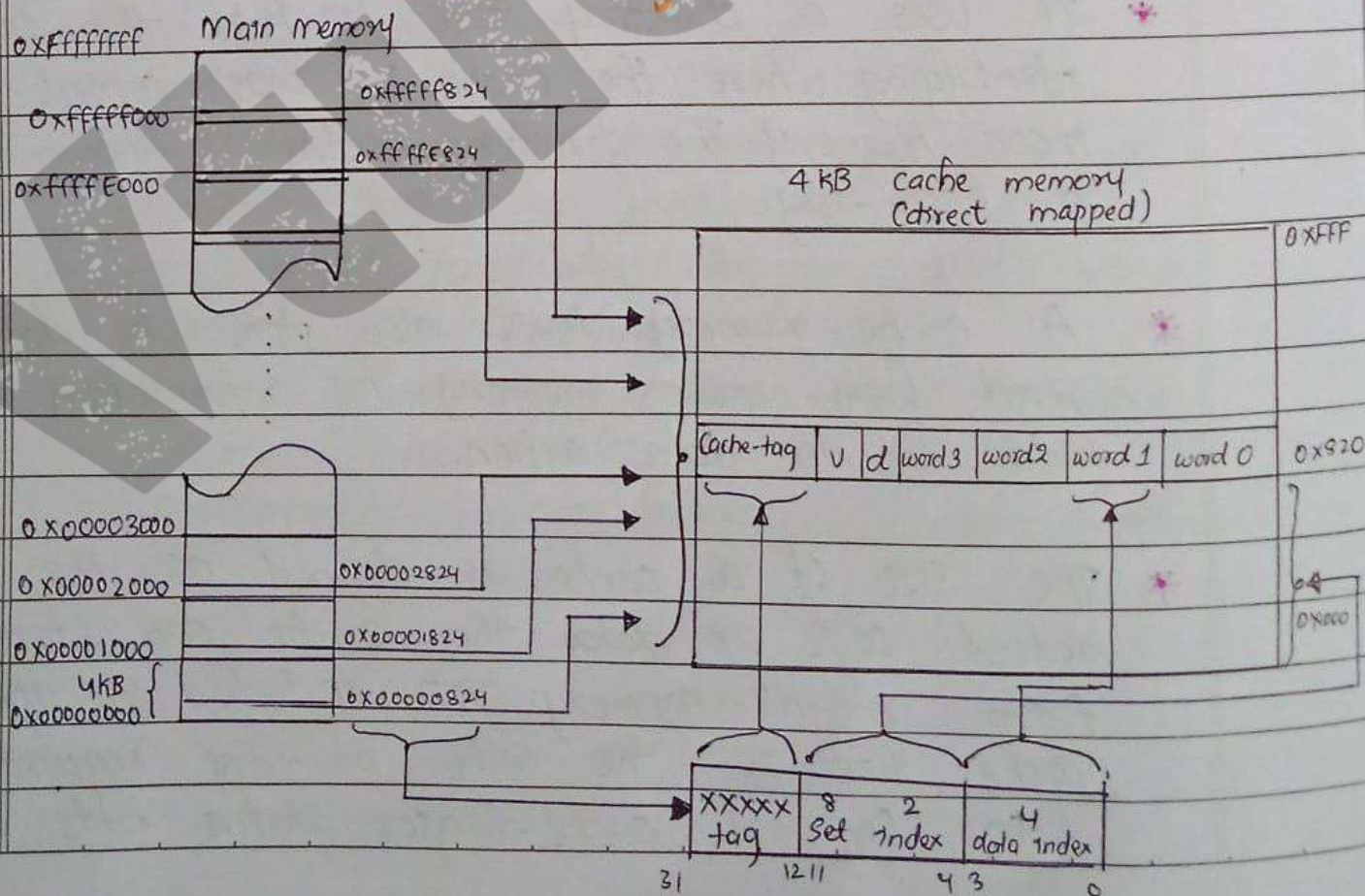
9) as Explain the basic architecture of cache memory.

- * A Simple cache memory is shown on the right side (A 4 kB cache consisting of 256 cache lines of four 32-bit words figure). It has three main parts: a directory store, a data section, a status information. All three parts of the cache memory are present for each cache line.
- * The cache must know where the information stored in a cache line originates from in main memory.
- * It uses a directory store to hold the address identifying where the cache line was copied from main memory. The directory entry is known as a cache-tag.
- * A cache memory must also store the data read from main memory. This information is held in the data section.
- * The size of a cache is defined as the actual code or data the cache can store from main memory. Not included in the cache size is the cache memory required to support cache-tags or status bits.

TAKE BREAK, BUT DON'T STOP

- * There are also status bits in cache memory to maintain state information. Two common status bits are the valid bit and dirty bit.
- * A valid bit marks a cache line as active, meaning it contains live data originally taken from main memory and is currently available to the processor core on demand.
- * A dirty bit defines whether or not a cache line contains data that is different from the value it represents in main memory.

b) Explain how main memory maps to a cache memory.



The figure represents the simplest form of cache, known as a direct-mapped cache.

- > In a direct-mapped cache each addressed location in main memory maps to a single location in cache memory.
- > Since main memory is much larger than cache memory, there are many addresses in main memory that map to the same single location in cache memory.
- > The figure shows this relationship for the class of addresses ending in $0x824$. The three bit fields introduced in A 4KB cache consisting of 256 cache lines of four 32-bit words figure.
- > The set index selects the one location in cache where all values in memory with an ending address of $0x824$ are stored.
- > The data index selected the word/halfword/byte in the cache line, in this case the second word in the cache line.
- > The tag field is the portion of the address that is compared to the cache-tag value found in the directory store.

→ In this example there are one million possible locations in main memory for every one location in cache memory

→ Only one of the possible one million values in the main memory can exist in the cache memory at any given time

→ The comparison of the tag with the cache-tag determines whether the request data is in cache or represents another of the million locations in main memory with an ending address of $0x824$.

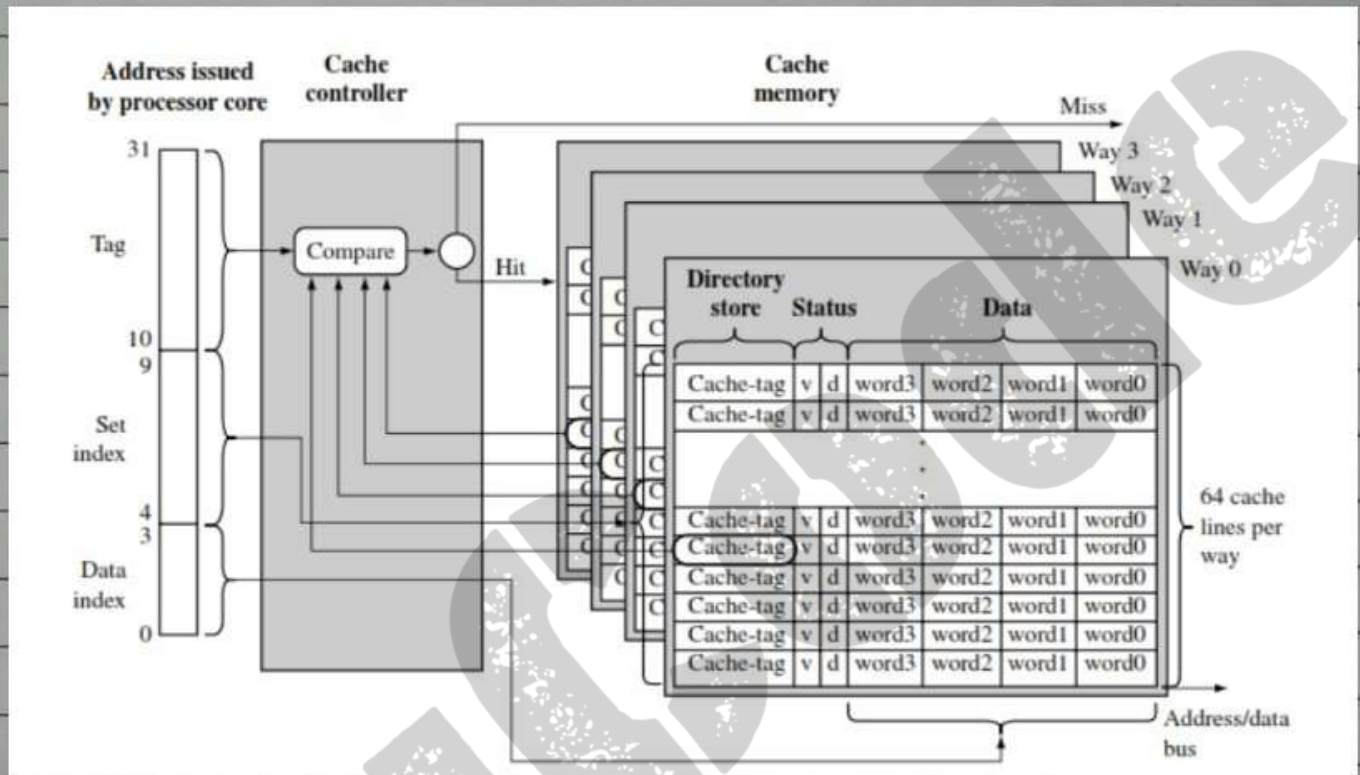
→ During a cache line fill the cache controller may forward the loading data to the core at same time it is copying it to cache, this is known as data streaming.

→ A direct-mapped cache is simple solution, but there is a design cost inherent in having a single location available to store a value from main memory.

→ Direct-mapped caches are subject to high levels of thrashing - a software battle for the same location in cache memory

OR

10) a) With a neat block diagram explain associative cache.



→ Some cache include an additional design feature to reduce the frequency of thrashing

→ This structural design feature is a change that divides the cache memory into smaller equal units, called ways.

→ The four cache lines with the same set index are said to be in the same set, which is the origin of the name "set index."

- In the above figure A 4 KB, four-way set associative cache. The cache has 256 total cache lines, which are separated into four ways, each containing 64 cache lines.
- The cache line contains four words. The set of cache lines pointed to by the set index are set associative.
- A data or code block from main memory can be allocated to any of the four ways in a set without affecting program behaviour. In other words, the storing of data in cache lines within a set does not affect program execution.
- Two sequential blocks from main memory can be stored as cache lines in the same way or two different ways.
- The important thing to note is that the data or code blocks from a specific location in main memory can be stored in any cache line that is a member of a set.
- The placement of values within a set is exclusive to prevent the same code or data block from simultaneously occupying two cache lines in a set.

b) Briefly explain cache line replacement policies.

- On a cache miss, the cache controller must select a cache line from the available set in cache memory to store the new information from main memory.
- The cache line selected for replacement is known as victim. If the victim contains valid, dirty data the controller must write the dirty data from the cache memory to main memory before it copies new data into the victim cache line.
- The process of selecting and replacing a victim cache line is known as eviction.
- The strategy implemented in a cache controller to select the next victim is called as its replacement policy.
- The replacement policy selects a cache line from the available associative member set; that is, it selects the way to use in the next cache line replacement.
- To summarize the overall process, the set index selects the specific cache line from the set to replace.

- ARM cached cores support two replacement policies, either pseudorandom or round-robin.
- Round-robin or cyclic replacement simply select the next cache line in a set to replace.
- when the victim counter reaches a maximum value, it is reset to a defined base value.
- Pseudorandom replacement randomly selects the next cache line in a set to replace.